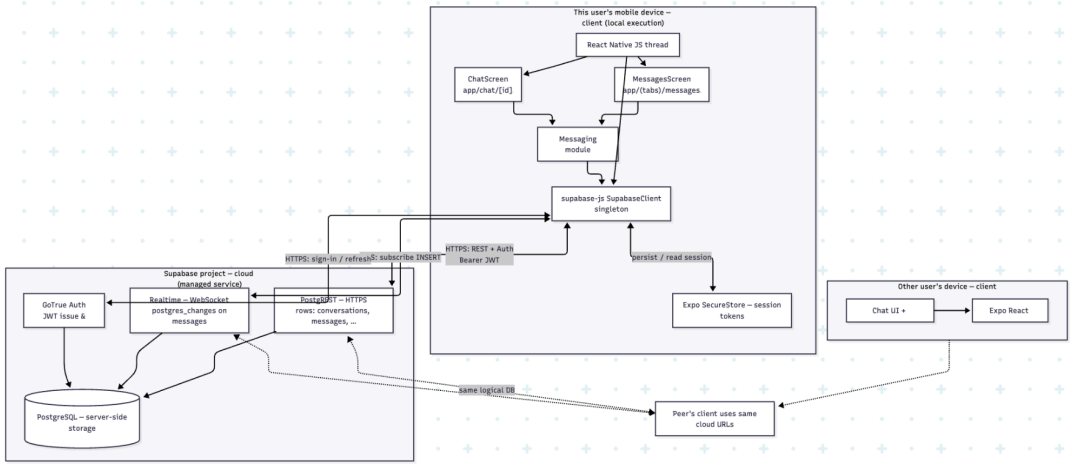
Noah’s User Story 2: In-App Messaging
User Story:
As a buyer and seller, I want to chat within the app regarding a specific listing so that I can coordinate pickup details safely without exchanging phone numbers.
Human Acceptance Criteria:
Messages appear instantly without requiring a manual page refresh.
Users receive a push notification when a message is received and the app is in the background.
Users can see a "Message Read" or "Delivered" indicator, so they aren't left wondering if the other party saw their pickup coordinates.
Machine Acceptance Criteria:
WebSocket or equivalent real-time connection successfully broadcasts messages between two authenticated users.
All chat messages are written to the database before being broadcast via WebSocket, ensuring no messages are lost if a user’s socket disconnects mid-chat.
Why this is important:
In-app messaging is a critical trust and safety feature that protects user privacy by eliminating the need to exchange personal phone numbers or rely on third-party apps. By facilitating instant, real-time communication directly tied to a specific listing, it significantly reduces friction when negotiating or arranging pickup logistics. This seamless communication layer—complete with read receipts and push notifications—provides users with peace of mind while keeping them actively engaged on the platform, ultimately fostering a safer and more reliable marketplace environment for both buyers and sellers.

The primary and secondary owners of the user story.

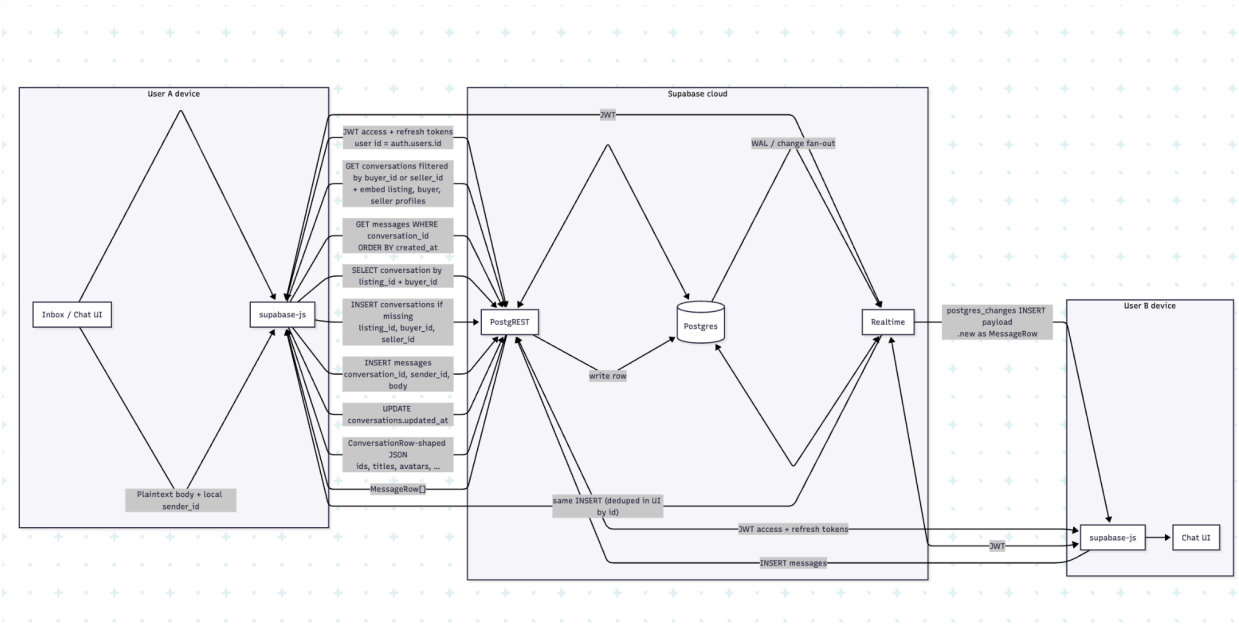
Primary Owner: Noah Choi
Secondary Owner: Jayden Lin

The date the code was merged into the main branch.
March 26, 2026

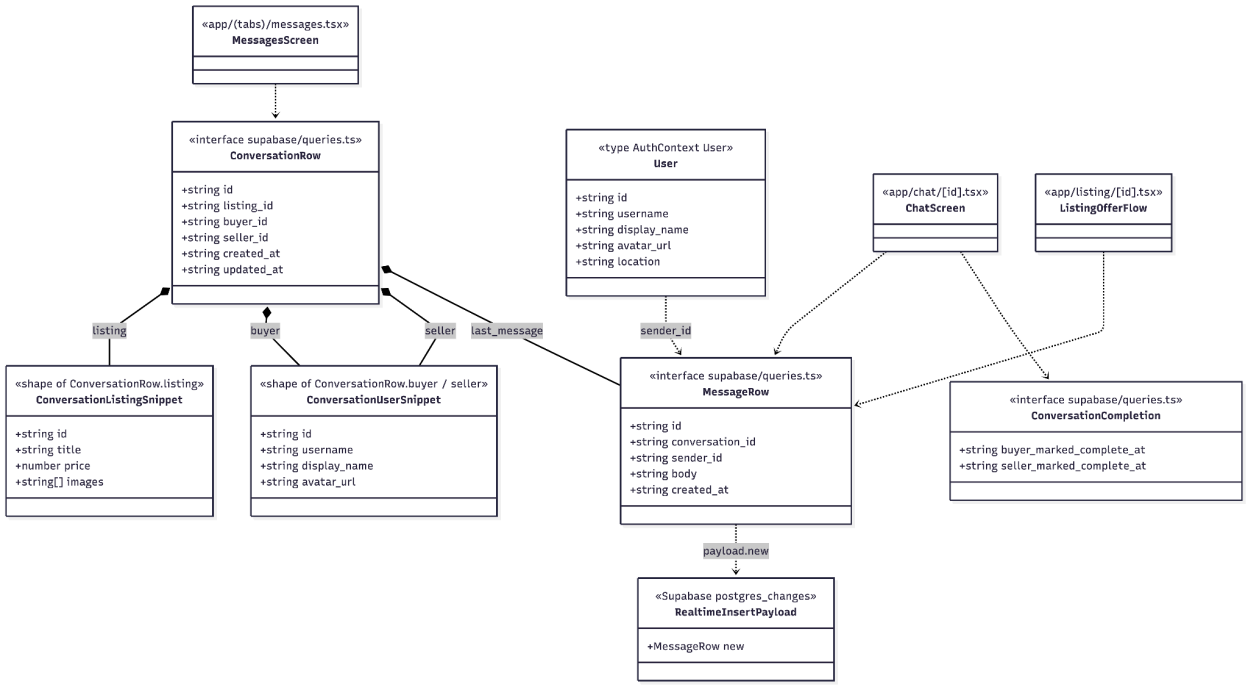
Architecture



Information flow



Class Diagram



Instant Messaging Implementation Reference

As noted in the original document, because this is a TypeScript/React Native codebase, "classes" here represent logical units (components, contexts, modules, and data shapes) since the code relies primarily on interfaces and functional components.

---

1. Relevant Classes and Implementation Units

`supabase/queries.ts` (Shared Data Shapes & API)

This module acts as the data layer contract, containing types and exported functions for messaging and ratings.

Public Fields and Methods

- **Identity**
  - `User`: Type alias containing `id`, `display_name`, `username`, `location`, `bio`, `avatar_url`, `email`. Represents the in-app profile snapshot tied to auth. The `id` is used as the `sender_id` or participant ID.
- **Thread List / Inbox Row**
  - `ConversationRow`: Contains `id`, `listing_id`, `buyer_id`, `seller_id`, `created_at`, `updated_at`. Core row identifiers and ordering logic.
  - `listing` (Nested on `ConversationRow`): Contains `id`, `title`, `price`, `images[]`. Listing preview for the thread.
  - `buyer` / `seller` (Nested on `ConversationRow`): Contains `id`, `username`, `display_name`, `avatar_url`. Counterparty display in the inbox.
  - `last_message`: Type `MessageRow | null`. Preview text and recency for the row.
- **Chat Bubble / History**
  - `MessageRow`: Contains `id`, `conversation_id`, `sender_id`, `body`, `created_at`. Represents one persisted chat message from Supabase.
- **Transaction + Rating**
  - `ConversationCompletion`: Contains `buyer_marked_complete_at`, `seller_marked_complete_at`. Timestamps for when each side marked the deal complete.
  - `UserRatingStats`: Contains `averageRating`, `ratingCount`. Read of denormalized profile stats.
- **Messaging & Completion API Methods**
  - `getOrCreateConversation(listingId, buyerId, sellerId)`: Ensures a `conversations` row for that listing + buyer; returns the conversation ID.

- fetchConversations(userId): Fetches threads where the user is a buyer or seller, joined with listing details, users, and the latest message preview.
- fetchMessages(conversationId): Fetches full message history in chronological order.
- sendMessage(conversationId, senderId, body): Inserts a new message and updates conversations.updated_at.
- subscribeToMessages(conversationId, onMessage): Subscribes to Realtime INSERT events on the messages table; returns an unsubscribe function.
- fetchConversationCompletion(conversationId): Reads transaction completion timestamps.
- markMyTransactionComplete(conversationId, role): Sets the completion timestamp for the acting role (buyer or seller).
- fetchMyRatingForConversation(conversationId, raterId): Checks if the current user has already submitted a rating for the thread.
- submitConversationRating(conversationId, raterId, rateeId, stars): Inserts a rating (1–5) into user_ratings.
- fetchUserRatingStats(userId): Reads average_rating and rating_count from the users table.

Private Fields and Methods

- **Internal Helpers**
  - isMissingUserRatingColumnsError(error): Internal helper method used to gracefully handle soft failures when rating columns are missing in the database.
- *Note: Interfaces and types contain no hidden/private fields as TypeScript interfaces represent purely public structural contracts.*

---

contexts/AuthContext.tsx (Auth Context)

Manages the authentication state across the application.

Public Fields and Methods

- **Context API**
  - AuthContext: The React context object. Default value is { user: null, setUser: noop }.
  - AuthProvider({ children }): Provider component that supplies user and setUser to the React tree.
  - useAuth(): Hook that returns the current { user, setUser } state.

Private Fields and Methods

- **Internal State & Types**

- AuthContextType: Module-local type defining the shape of the context. Not exported.
- useState<User | null>: The internal React state inside AuthProvider that actually backs the context value.

---

supabase/client.ts (Supabase Client)

Initializes and exports the connection to the backend database.

Public Fields and Methods

- **Database Connection**
  - supabase: The initialized SupabaseClient instance. Configured with an ExpoSecureStoreAdapter (which uses getItem, setItem, removeItem) for secure JWT session persistence.

Private Fields and Methods

- **Internal Configuration**
  - Default URL and anonymous key fallbacks used internally when environment variables are unset (e.g., during local development).

---

app/chat/[id].tsx (ChatScreen)

The full-screen conversational UI for a single listing thread.

Public Fields and Methods

- **Screen Component & Routing**
  - ChatScreen(): The default exported component rendering the chat interface.
  - Route parameters (via useLocalSearchParams): Accepts id (listing), buyerId?, sellerId, usernames, listing title/price/image, and optional mock flags.

Private Fields and Methods

- **Routing & Navigation**
  - useRouter, useLocalSearchParams, useSafeAreaInsets: Hooks for managing navigation, parameters, and device safe zones.
- **Theme & Layout**
  - useColorScheme, Colors, Brand, StyleSheet: Hooks and constants for styling the UI.

- **Internal State**
  - Variables including: conversationId, messages, input, loading, sending, completion, ratingStars, txnMetaLoading, finishingTxn, showRatingModal, ratingSubmitting, flatListRef.
- **Derived State**
  - Calculated variables: effectiveBuyerId, effectiveSellerId, amBuyer, myRole, rateeHandle, headerPeerHandle, iMarked, otherMarked, bothMarkedComplete, hasRated, ratingEligible, rateeId.
- **Effects (Lifecycle)**
  - Effects to load the thread, subscribe/unsubscribe to Supabase Realtime, and handle auto-scrolling to the latest message.
- **Event Handlers**
  - refreshTransactionMeta: Updates transaction status.
  - handleSend: Processes outgoing message text.
  - handleFinishTransaction: Triggers the transaction completion flow.
  - handleSubmitRating: Submits user rating to the backend.
  - renderMessage: UI render function for individual chat bubbles.

---

app/(tabs)/messages.tsx (MessagesScreen)

The main inbox view, separating active chats by Buying and Selling.

Public Fields and Methods

- **Screen Component**
  - MessagesScreen(): The default exported component rendering the dual-tab inbox.

Private Fields and Methods

- **Internal State**
  - activeTab: Tracks whether "Buying" or "Selling" is selected.
  - conversations: The raw list of loaded inbox threads.
  - loading: Tracks the fetching state.
- **Data Fetching**
  - loadConversations: Fetches inbox rows from the API.
  - useFocusEffect: Refreshes the inbox list whenever the screen gains focus.
- **Derived State**
  - buyingChats, sellingChats, currentChats: Filtered lists based on the active tab and user role.
- **UI Handlers**
  - renderConversation: Renders an individual inbox row and handles navigation routing to specific ChatScreen parameters.

`app/listing/[id].tsx` (ListingDetailScreen)

The detail view for a specific item listing, which initiates the messaging flow.

Public Fields and Methods

- **Screen Component**
  - `ListingDetailScreen()`: Default export for the listing view.
- **Messaging Integration**
  - `handleMakeOffer`: Triggered to start a chat. Calls `getOrCreateConversation` and `sendMessage` to send an automated first offer.

Private Fields and Methods

- **Internal State & UI**
  - Variables tracking `post`, `loading`, modals, boost timers, `loadError`.
  - `loadPost`: Fetches the listing data.

`components/chat/TransactionRatingModal.tsx` (TransactionRatingModal)

UI component for submitting a 1-5 star rating upon transaction completion.

Public Fields and Methods

- **Component & Props**
  - `TransactionRatingModal`: The exported component.
  - Props: `visible`, `otherUsername`, `busy`, `onClose`, `onSubmit(stars)`, optional `subtitle`. (Note: The parent component owns and executes the actual API calls).

Private Fields and Methods

- **Internal State & UI**
  - `stars`: Local state tracking the currently selected star amount.
  - `useEffect` hooks to reset the UI state when the modal opens/closes.
  - Styles and rendering logic for the interactive star row.

---

2. Third-Party Technologies, Libraries, and APIs

Below is the comprehensive list of external tools utilized in the messaging implementation. Versions are mapped strictly to standard dependencies (via `package.json`).

Technology	Version	Purpose	Rationale	Source, Author & Documentation
TypeScript	~5.9.2	Core language and static typing.	Provides safer refactoring and strict type contracts compared to plain JS.	**Author:** Microsoft **Source:** GitHub **Docs:** typescriptlang.org/docs
JavaScript	ES / Metro	Runtime target.	Standard execution environment for React Native.	**Author:** ECMAScript **Docs:** MDN Web Docs
React	19.1.0	UI declarative rendering, hooks, and context.	Industry standard for component-based architecture in RN.	**Author:** Meta **Source:** GitHub

				Docs: react.dev
React Native	0.81.5	Native UI primitives and runtime.	Allows building cross-platform mobile apps using React.	**Author:** Meta **Source:** GitHub **Docs:** reactnative.dev
Expo SDK	~54.0.33	Development tooling, native modules, and CLI.	Drastically speeds up development over raw RN by managing native builds.	**Author:** Expo **Source:** GitHub **Docs:** docs.expo.dev
Expo Router	~6.0.23	File-based routing and deep linking.	Simplifies navigation via an app/ directory (similar to Next.js).	**Author:** Expo **Docs:** Router Intro

@supabase/supabase-js	^2.100.0	REST, Realtime messaging, and Auth client.	Provides an all-in-one Backend-as-a-Service, eliminating the need for a custom Node backend.	**Author:** Supabase **Source:** GitHub **Docs:** supabase.com/docs
Supabase Platform	Cloud / DB	Postgres, PostgREST, Realtime sockets, GoTrue auth.	Managed infrastructure with built-in Row Level Security (RLS).	**Author:** Supabase **Docs:** DB, Realtime
PostgREST	Via Supabase	Translates HTTP requests to SQL queries.	Generates an auto-updating REST API directly from the Postgres schema.	**Author:** PostgREST **Docs:** postgrest.org
expo-secure-store	~15.0.8	Secure auth session persistence.	Encrypts JWTs natively, far more secure than plain text	**Author:** Expo

			AsyncStorage.	**Docs:** SecureStore Docs
react-native-url-polyfill	^3.0.0	Injects Web URL APIs into React Native.	The Supabase JS client requires native Web URL parsing behaviors to function.	**Author:** Community **Source:** GitHub
React Navigation Suite (@react-navigation/native, @react-navigation/bottom-tabs, @react-navigation/elements)	^7.1.8 ^7.4.0 ^2.6.3	Core navigation, focus effects, and bottom tab UX.	The community standard routing engine that underpins Expo Router.	**Author:** React Navigation **Source:** GitHub **Docs:** reactnavigation.org
react-native-safe-area-context	~5.6.0	Handles safe area insets.	Prevents UI elements from hiding behind device notches and home indicators.	**Author:** Software Mansion **Source:** GitHub

react-native-screens	~4.16.0	Native screen container handling.	Optimizes memory and performance for navigation stacks.	**Author:** Software Mansion **Source:** GitHub
react-native-gesture-handler	~2.28.0	Advanced touch interactions.	Required dependency for React Navigation and Expo Router.	**Author:** Software Mansion **Docs:** Gesture Handler Docs
react-native-reanimated & react-native-worklets	~4.1.1 0.5.1	Native UI thread animations.	Ensures smooth 60fps animations for chat modals and routing transitions.	**Author:** Software Mansion **Docs:** Reanimated Docs
expo-image	~3.0.11	Image rendering.	Provides superior caching and performance for chat avatars and listing images over standard RN images.	**Author:** Expo **Docs:** Image Docs

@expo/vector-icons	^15.0.3	Bundled vector icon support.	Pre-installed icon sets optimized for Expo environments.	**Author:** Expo **Docs:** Icons Docs
@react-native-async-storage/async-storage	2.2.0	Key-value string persistence.	Standard unencrypted local storage for non-sensitive app settings.	**Author:** Community **Source:** GitHub **Docs:** Async Storage Docs
Expo Utils (expo-constants, expo-linking, expo-web-browser, expo-auth-session)	Various ~18/~15/~8	App config, deep links, and OAuth flows.	Provides unified cross-platform hooks to tap into device-level features.	**Author:** Expo **Docs:** Expo API Ref
react-native-web & react-dom	~0.21.0	Web compilation targets.	Allows the RN codebase to compile and run in browser	**Author:** Meta / Community

	19.1.0		environments.	**Source:** RN Web GitHub
react-native-get-random-values	^2.0.0	Crypto-safe random generation.	Required polyfill for Supabase and modern crypto operations in RN.	**Author:** Community **Source:** GitHub
lucide-react-native	^1.6.0	Custom icon set.	Provides clean, consistent SVG-based icons for UI elements.	**Author:** Lucide **Docs:** lucide.dev
Node.js & **npm**	LTS	Development environment, runtime, and package management.	Standard ecosystem for JavaScript-based development.	**Author:** OpenJS Foundation / npm **Docs:** nodejs.org
Metro Bundler	Via Expo	JS code bundler.	Default and fastest bundler purpose-built for React Native.	**Author:** Meta

				Docs: metrobundler.dev
ESLint & eslint-config-expo	^9.25.0 ~10.0.0	Static code analysis.	Catches syntax/style errors early; aligned with Expo community rules.	**Author:** OpenJS / Expo **Source:** GitHub **Docs:** eslint.org
Jest & ts-jest	^30.3.0 ^29.4.6	Unit testing and TS transforms .	The default and most stable testing framework for RN applications.	**Author:** OpenJS / Community **Source:** GitHub **Docs:** jestjs.io
@types/react	~19.1.0	React type definitions.	Allows TypeScript compiler to understand React primitives.	**Author:** DefinitelyTyped

				Source: npm
dotenv	^17.3.1	Local environment variables.	Safely loads API keys from local .env files during tooling execution.	**Author:** Community **Source:** GitHub
ts-node	^10.9.2	TypeScript execution in Node.	Allows running utility scripts and tests without pre-compiling to JS.	**Author:** Community **Source:** GitHub
@expo/ngrok	^4.1.3	Secure tunneling.	Allows remote physical devices to hit the local dev server over the internet.	**Author:** Expo **Docs:** See Expo CLI documentation.

Instant messaging — database storage and failure-mode effects (Buyinz)

Scope: long-term persistence that backs instant messaging (threads, messages), listing-attached conversations, and transaction completion / peer ratings as implemented against Supabase Postgres. Sizes are approximate on-disk tuple widths for planning; real storage includes indexes, TOAST, WAL, page fill, and free space map (often 1.5×–3× row data for active tables).

Sizing method (PostgreSQL heap tuples, simplified):

- Fixed types: `uuid` 16 B; `timestamptz` 8 B; `int4` 4 B; `int8` 8 B; `smallint` 2 B; `boolean` 1 B (plus alignment padding in the row).
- `numeric(4,2)`: often ~12–16 B for small values (variable length in Postgres).
- `text` / `varchar` / `jsonb`: payload UTF-8 length + ~1–4 B varlena header per column (long values may be TOASTed to an extension table—still your disk cost).
- Add ~24 B per row for the heap tuple header (version, NULL bitmap base, etc.) as a rule of thumb.
- NULL columns cost little (usually one bit in the NULL bitmap).

Exact bytes require `pg_column_size(row)` on live data; below uses ranges (min plausible short content vs typical content).

---

1. Data types stored in the database (messaging-relevant)

Table: `public.conversations`

One row per (listing, buyer) pair: the thread container for all messages about that sale.

Field	SQL type (typical)	Purpose	Size contribution (approx.)
`id`	`uuid` PK	Stable thread id; referenced by	16 B

		`messages`, `user_ratings`.	
`listing_id`	`uuid` FK → `posts.id`	Which listing this chat is about.	16 B
`buyer_id`	`uuid` FK → auth/public user	Buyer participant (auth user id).	16 B
`seller_id`	`uuid` FK → auth/public user	Seller participant.	16 B
`created_at`	`timestamptz`	When the thread was opened.	8 B
`updated_at`	`timestamptz`	Bumped when a message is sent (inbox ordering / freshness).	8 B
`buyer_marked_complete_at`	`timestamptz` nullable	When buyer tapped “transaction complete.”	0–8 B
`seller_marked_complete_at`	`timestamptz` nullable	When seller did the same.	0–8 B

Estimated row size: ~110–140 B fixed-width core + header → round to ~150–200 B per thread before indexes. With typical 2–3 secondary indexes (e.g. buyer, seller, listing), plan ~400–800 B effective disk per conversation in a warm database.

Table: `public.messages`

One row per chat message persisted for a conversation.

Field	SQL type (typical)	Purpose	Size contribution (approx.)
`id`	`uuid` PK	Deduplication in UI, Realtime payload key.	16 B

`conversation_id`	`uuid` FK	Which thread this belongs to.	16 B
`sender_id`	`uuid`	Who wrote it (matches authenticated user).	16 B
`body`	`text`	Message content (plain text in app).	Highly variable: ~50 B–200 B (short) to several KB (long paste). App caps input in UI, but DB should assume abuse up to configured max.
`created_at`	`timestamptz`	Sort order, timestamps in bubbles.	8 B

Estimated row size: ~90 B fixed + body + header → ~150–350 B typical short message; 1–8 KB possible if large bodies allowed.

Index-heavy: conversations usually query by `conversation_id` + `created_at` → additional ~½×–1× row size per index entry in practice.

Table: `public.user_ratings`

One row per (conversation, rater): peer star rating after mutual completion (or your policy as implemented).

Field	SQL type	Purpose	Size (approx.)
`id`	`uuid` PK	Rating row id.	16 B
`conversation_id`	`uuid` FK	Which deal is being rated.	16 B

`rater_id`	`uuid` FK → `auth.users`	Who gave the stars.	16 B
`ratee_id`	`uuid` FK → `auth.users`	Who was rated.	16 B
`stars`	`smallint` (1–5)	Numeric rating.	2 B
`created_at`	`timestamptz`	Audit / ordering.	8 B

Estimated row size: ~90–120 B + header → ~120–180 B per rating; unique (conversation_id, rater_id) adds index overhead.

Table: `public.users` (app profile, not `auth.users`)

Denormalized profile for display in inbox/chat joins and average rating cache.

Field	SQL type (typical)	Purpose	Size (approx.)
`id`	`uuid` PK (= auth user)	Join key to buyer/seller/message sender.	16 B
`display_name`	`text`	Shown in UI.	~30–200 B typical
`username`	`text` unique	Handle, routing, upsert key in app.	~20–80 B
`location`	`text`	ZIP / area string.	~5–64 B
`bio`	`text` nullable	Profile blurb.	0–2 KB
`avatar_url`	`text` nullable	Remote image URL.	0–300 B
`average_rating`	`numeric(4,2)`	Cached mean from `user_ratings`.	~12–16 B
`rating_count`	`integer`	Cached count of ratings received.	4 B

Estimated row size: typically ~500 B–3 KB per user depending on bio/URLs; indexes on `username`, `id` add more.

(If your project adds `email` on `public.users`, include it similarly; the current app often uses `AuthContext` email separately from the row shape in code.)

Table: `public.posts` (listings referenced by conversations)

A conversation points at a post as the “listing id.” Listing rows exist independent of chat but are storage the messaging story depends on.

Field	SQL type (typical)	Purpose	Size (approx.)
`id`	`uuid` PK	`conversations.listing_id` target.	16 B
`user_id`	`uuid`	Listing owner (seller).	16 B
`type`	`text` / enum	`sale` vs `iso`, etc.	~4–16 B
`title`	`text`	Shown in chat header / inbox.	~30–200 B
`description`	`text` nullable	Body copy.	0–4 KB
`price`	`numeric` / `double`	Ask price for offers.	~8–16 B
`condition`	`text` nullable	Quality label.	~10–32 B
`category`	`text`	Taxonomy.	~10–40 B

`images`	`jsonb` / `text[]`	Array of image URLs.	~200 B–8 KB (many URLs)
`hashtags`	`jsonb` / `text[]`	Tags.	~50–50 0 B
`created_at`	`timestamptz`	Feed ordering.	8 B
`sold`	`boolean`	Listing state (migration).	1 B
`boosted_until`	`timestamptz` nullable	Pro boost window.	0–8 B

Estimated row size: ~600 B–12 KB typical depending on description + image JSON.

Table: `auth.users` (Supabase-managed; long-term identity)

Not in `public`, but messages.sender_id and FKs align with these ids.

Concept	Purpose
`id` (`uuid`)	Canonical user id used across `public.users`, `messages`, `conversations`, ratings.
`email`, encrypted password / provider ids	Sign-in; not your app's message body.

Byte estimate: managed by Supabase; plan ~0.5–2 KB per auth user for common email/password metadata plus security overhead you don't size manually.

Optional: indexes (not separate "data types" but disk)

Messaging hot paths use indexes such as `messages(conversation_id, created_at)`, `conversations(buyer_id)`, `conversations(seller_id)`,

`user_ratings(ratee_id)`. Rule of thumb: add 50–150% of hot table row volume for OLTP indexes until you measure.

---

2. Frontend / system failure and abuse — user-visible vs internal effects

Legend:

- User-visible: what a person sees (UI copy, empty states, errors, stale data).
- Internal: logs, metrics, state machine, retries, support tickets, infra alerts—often no or partial user explanation.

Scenario	User-visible effects	Internal / engineering effects
App process crashed	Current screen disappears iOS/Android may relaunch app; unsent draft in RAM lost unless you add local persistence; after relaunch, re-fetch from network shows last saved server state.	OS crash report; session usually restored from SecureStore on next launch if Supabase client rehydrates cleanly.

Lost all runtime state (RAM) but process restarts cleanly	Same as cold start: brief loading spinners; inbox/chat reload from API. In-flight send may have completed server-side even if UI didn't get response—user may see duplicate concern; dedupe uses message `id`.	Hooks re-run; Realtime resubscribes; verify idempotency for "tap send twice."
Erased all stored data (uninstall app / "Clear data" / wiped SecureStore)	Signed out; must sign in again; no local cache—everything from server after auth.	New device install path; push tokens (if any) invalid until re-registered.
DB data appears corrupt (null ids, wrong FK, garbled text, broken JSON)	Broken threads: missing listing or peer names, empty bubbles, errors from PostgREST (400/500), "Could not	DB constraints should reject bad inserts; if corruption already committed—manual SQL repair, restore from backup, app may need defensive null checks.

	load conversation” alerts. Ratings may show 0 or fail if numerics invalid.	
Remote procedure call failed (PostgREST / Realtime RPC, e.g. `insert`/`select` error)	Inline Alert or silent failure in logs depending on code path; send may restore draft on failure (chat does for send).	HTTP status + Supabase error code in logs; retry policy; monitor 4xx/5xx rates.
Client overloaded (CPU-bound UI thread)	Jank, delayed taps, keyboard lag, slow scroll; feels “frozen.”	JS thread profiling; reduce re-renders; pagination for long threads.
Client out of RAM	iOS/Android may kill the app (same as crash); `OOM` before kill: extreme lag, then abrupt exit.	Low-memory warnings on native; shrink image cache; avoid loading huge message lists at once.
Database out of space	Writes fail: cannot send message, cannot mark complete, cannot rate; user sees	Postgres panics or read-only mode; disk alerts; VACUUM, expand volume, delete old data/archival.

	generic errors or timeouts. Reads may still work briefly.	
Lost network connectivity	Timeouts, “failed to send,” offline UI if you add it (current app: mostly error alerts); Realtime disconnects —no live updates until reconnect; messages queue only if you implement offline queue (not default).	Supabase client reconnect backoff; log connectivity state; consider exponential backoff.
Lost access to database (wrong URL/key, project paused, RLS denies all, region outage)	Blank inbox, perpetual loading, auth may work but data calls fail.	Incidents on Supabase status; API key rotation; RLS policy bugs; on-call.
Bot signs up and spams users	Legitimate users get flood of messages / threads; inbox	Rate limits (Supabase Auth, edge functions), CAPTCHA, phone verify, RLS (who can insert into `messages`), report/block tables, moderation job, IP/abuse policies.

	unusable; trust drops.	